Python Topic 20 — Concurrency: Threading, Multiprocessing & the GIL

Full Stack Python + AI Roadmap • 3 layers: New to Python → Intermediate → Expert • Completes the concurrency picture (with async, Topic 18)

🌱 Layer 1 — New to Python

Your program sometimes needs to do **multiple things at once**. Python gives three ways, and the right one depends on *what kind* of work you're doing.

A kitchen analogy:

- **Threading** — one chef juggling several dishes, switching while things simmer. Great when the chef is mostly *waiting* (water boiling, oven baking).
- **Multiprocessing** — hiring *multiple chefs*, each with their own station, all cooking simultaneously. Great when there's real *chopping* to do (actual work, not waiting).
- **async** (Topic 18) — one very organized chef who never stands idle, always doing something useful while other things wait.

ℹ️ **One-line rule:** waiting → threads or async; heavy computation → processes.

👩‍💻 Layer 2 — Intermediate (real code + how it works)

Threading — for I/O-bound work

```
import threading

def download(name):
    print(f"Downloading {name}")
    # ... waits on network ...

threads = []
for name in ["a", "b", "c"]:
    t = threading.Thread(target=download, args=(name,))
    t.start()               # start the thread
    threads.append(t)

for t in threads:
    t.join()                # wait for all to finish
```

Threads share the same memory and are lightweight. Good when tasks spend time *waiting* (downloads, API calls, file reads).

Multiprocessing — for CPU-bound work

```
from multiprocessing import Process

def crunch(n):
    total = sum(i**2 for i in range(n))   # heavy CPU work
    print(total)

processes = []
for _ in range(4):
    p = Process(target=crunch, args=(10**7,))
    p.start()               # each runs on a separate CPU core
    processes.append(p)

for p in processes:
    p.join()
```

Each process is a **separate Python interpreter with its own memory**, running on its own CPU core — so they compute in true parallel. The cost: processes are heavier to start and don't share memory easily.

The modern, easier way — `concurrent.futures`

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# For I/O-bound work → threads
with ThreadPoolExecutor(max_workers=5) as executor:
    results = executor.map(download, ["a", "b", "c"])

# For CPU-bound work → processes
with ProcessPoolExecutor(max_workers=4) as executor:
    results = executor.map(crunch, [10**7, 10**7, 10**7])
```

✅ **Use this in practice:** one clean API for both — swap `ThreadPoolExecutor` for `ProcessPoolExecutor` depending on the work type.

> 🔑 `concurrent.futures` — Deep Dive

The name decodes into two ideas: **concurrent** (running tasks in overlapping time) and **futures** (a placeholder for a result that isn't ready *yet* but *will be*). A **future** is an object representing "a result still being computed" — you get it immediately on submission, and the value fills in later. If you know JS Promises, a future is Python's version of a promise.

The module is a **high-level wrapper** over threading and multiprocessing: instead of manually creating `Thread` / `Process` objects, calling `.start()` / `.join()`, and tracking them in lists, you get a clean pool-based API. Two classes, **same API**: `ThreadPoolExecutor` (I/O-bound) and `ProcessPoolExecutor` (CPU-bound).

Way 1 — `.map()` (run one function over many inputs)

```
from concurrent.futures import ThreadPoolExecutor

def square(n):
    return n * n

with ThreadPoolExecutor(max_workers=3) as executor:
    results = executor.map(square, [1, 2, 3, 4, 5])

print(list(results))    # [1, 4, 9, 16, 25]  (results come back in input order)
```

`executor.map()` works like the built-in `map()`, but runs the calls concurrently across the pool.

Way 2 — `.submit()` (individual control + the actual Future)

```
from concurrent.futures import ThreadPoolExecutor

def slow_task(n):
    return n * 2

with ThreadPoolExecutor(max_workers=3) as executor:
    future = executor.submit(slow_task, 10)   # returns a Future immediately

    print(future.done())      # False — not finished yet
    result = future.result()  # BLOCKS here until the result is ready
    print(result)             # 20
```

Here the "future" is explicit: `.submit()` hands you a `Future` right away. Ask `.done()` (is it finished?) or `.result()` (give me the value, waiting if needed).

Handling many futures as they finish — `as_completed()`

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch(n):
    return f"result-{n}"

with ThreadPoolExecutor(max_workers=3) as executor:
    futures = [executor.submit(fetch, i) for i in range(5)]
```

```
    for future in as_completed(futures):   # yields each as it finishes
        print(future.result())
```

`as_completed()` yields results **in the order they finish**, not the order submitted — useful when some tasks are faster than others.

Why it exists — raw vs. futures

```
# ❌ Raw threading — manual bookkeeping, return values are awkward
import threading
threads = []
for i in range(5):
    t = threading.Thread(target=work, args=(i,))
    t.start()
    threads.append(t)
for t in threads:
    t.join()

# ✅ concurrent.futures — clean, and return values come free
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=5) as executor:
    results = list(executor.map(work, range(5)))
```

✅ **Summary:** `concurrent.futures` is a high-level, unified API for running tasks in a pool of threads or processes, where each task hands back a "future" (a placeholder for its eventual result). The `with` block auto-manages the pool (creates workers, waits, cleans up) and gives you return values easily — which is painful with raw threads. It's the modern, practical tool; you rarely touch raw `Thread` / `Process` once you know it.

> 🪝 **Sub-hook:** *A future = a result-to-be (like a JS promise).* `ThreadPoolExecutor` *for I/O,* `ProcessPoolExecutor` *for CPU — same API, swap one word.* `.map()` *for bulk,* `.submit()` *+* `.result()` *for control,* `as_completed()` *for "handle each as it finishes."*

🧠 Layer 3 — Expert (the GIL & the real trade-offs)

1. The GIL — the single most important concept here

The **GIL (Global Interpreter Lock)** is a lock inside CPython that allows **only one thread to execute Python bytecode at a time** — even on a multi-core machine. So Python threads don't run Python code in true parallel.

```
# Two threads doing CPU work → NO speedup (GIL lets only one run at a time)
# Two threads doing I/O work → BIG speedup (a thread releases the GIL while waiting)
```

Why threading still helps for I/O: when a thread hits a blocking I/O call (network, disk), it **releases the GIL**, letting another thread run. During I/O the thread isn't executing Python bytecode anyway — it's waiting. So threads overlap

their *waiting*, which is the win.

Why threading fails for CPU work: CPU-bound code never stops executing bytecode, so it never releases the GIL. Two CPU threads just take turns — no faster than one, plus overhead.

2. How each model sidesteps (or doesn't) the GIL

Model	Parallelism	GIL impact	Best for
asyncio	Concurrency, 1 thread	N/A (single thread)	I/O-bound, many tasks
threading	Concurrency, many threads	Limited by GIL	I/O-bound
multiprocessing	True parallelism	Bypasses GIL (separate interpreters)	CPU-bound

Multiprocessing beats the GIL because **each process has its own interpreter and its own GIL** — so 4 processes genuinely use 4 cores.

3. The decision framework (interview gold)

```
Is the work I/O-bound (waiting on network/disk/DB)?
├── Yes, and lots of tasks / modern code  → asyncio
├── Yes, simpler or using blocking libs   → threading
└── No — it's CPU-bound (computation)      → multiprocessing
```

4. Thread safety — the classic bug

```python
import threading

counter = 0
def increment():
    global counter
    for _ in range(100000):
        counter += 1     # ❌ NOT atomic — race condition!

# Two threads → final counter is often WRONG (lost updates)
```

Because threads share memory, two threads modifying the same variable can interleave and corrupt it. Fix with a **lock**:

```python
lock = threading.Lock()
def increment():
    global counter
    for _ in range(100000):
        with lock:           # only one thread in here at a time
            counter += 1
```

⚠️ **Note:** multiprocessing largely avoids this — separate memory means no shared-state races — but then you need explicit ways to pass data between processes (`Queue` , `Pipe` , shared memory).

5. Passing data between processes

```
from multiprocessing import Process, Queue

def worker(q):
    q.put("result")          # send data back to the parent

q = Queue()
p = Process(target=worker, args=(q,))
p.start()
print(q.get())               # "result"
p.join()
```

Since processes don't share memory, you communicate through a `Queue` or `Pipe` . Data is *pickled* (serialized) to cross the process boundary — which is why process overhead is higher.

6. The GIL is changing — Python 3.13+

Recent Python has an experimental **"free-threaded" build (PEP 703)** that can disable the GIL, allowing true multi-core threading. It's optional and still maturing as of 2026, but it's a forward-looking point that shows depth if it comes up in an interview.

7. Where this fits your actual work

ℹ️ **For FastAPI + AI backends:** you'll mostly use **async** (I/O-bound: LLM calls, DB). You rarely write raw threads/processes because uvicorn/gunicorn run multiple **worker processes** for you (multiprocessing under the hood — using all cores), and heavy background jobs go to **Celery** (Phase 5). So this topic is mostly about understanding what's happening underneath and answering interview questions — not day-to-day code.

🎯 Interview Q&A Cards

Q: What is the GIL and why does it matter?

A: The Global Interpreter Lock lets only one thread execute Python bytecode at a time in CPython. So threads don't speed up CPU-bound work, but they help I/O-bound work because a thread releases the GIL while waiting on I/O.

Q: Threading vs multiprocessing vs asyncio — when each?

A: asyncio and threading for I/O-bound work (waiting on network/disk/DB); asyncio scales to many tasks on one thread, threading suits blocking libraries. Multiprocessing for CPU-bound work — separate interpreters give true parallelism across cores.

Q: Why does multiprocessing bypass the GIL?

A: Each process is a separate Python interpreter with its own GIL and memory, so processes truly run in parallel on multiple cores. The trade-off is higher startup cost and data must be serialized (pickled) to pass between them.

Q: What's a race condition and how do you prevent it?

A: When threads share memory and interleave non-atomic operations, updates get lost or corrupted. Prevent it with a lock (`threading.Lock`) so only one thread enters the critical section at a time.

🪝 **Memory hook:** *GIL = one thread runs Python at a time. I/O-bound → threads/async (waiting releases the GIL). CPU-bound → processes (own interpreter, real parallelism). Threads share memory (need locks); processes don't (need queues).*

✅ **One-liner for interviews:** "Python has three concurrency models: asyncio (single-thread, cooperative, I/O-bound), threading (I/O-bound, limited by the GIL), and multiprocessing (CPU-bound, true parallelism). The GIL lets only one thread run Python bytecode at a time, so threads don't speed up CPU work — but they help I/O because a thread releases the GIL while waiting. For CPU-bound work you use multiprocessing, since each process has its own interpreter and GIL."